

HW1 report

1. 設計方式

變數方面我額外制定三個用來記錄狀態、資料。

```
logic [1:0] condition_n0;  
logic [1:0] condition_n1;  
logic [1:0] condition_n2;  
logic [1:0] condition_n3;  
logic [1:0] condition_n4;
```

Condition 用來記錄 out 前兩個 bit 的狀態:

1. 00 表示 out 沒跟任何 in 銜接
2. 01 表示 out 只和一個 in 銜接
3. 10 表示 out 和兩個或兩個以上的 in 銜接

```
logic [15:0] data_n0;  
logic [15:0] data_n1;  
logic [15:0] data_n2;  
logic [15:0] data_n3;  
logic [15:0] data_n4;
```

Data 用來儲存 out 內 16bit 值的變化

```
logic conflict_n0;  
logic conflict_n1;  
logic conflict_n2;  
logic conflict_n3;  
logic conflict_n4;
```

Conflict 用 1bit 記錄 in 傳進 out 中時可能的狀態

1. 0 表示可能沒傳或者是遇到優先級更高的 in 導致資料傳不進去
2. 1 表示傳入成功，沒受到多餘的干擾

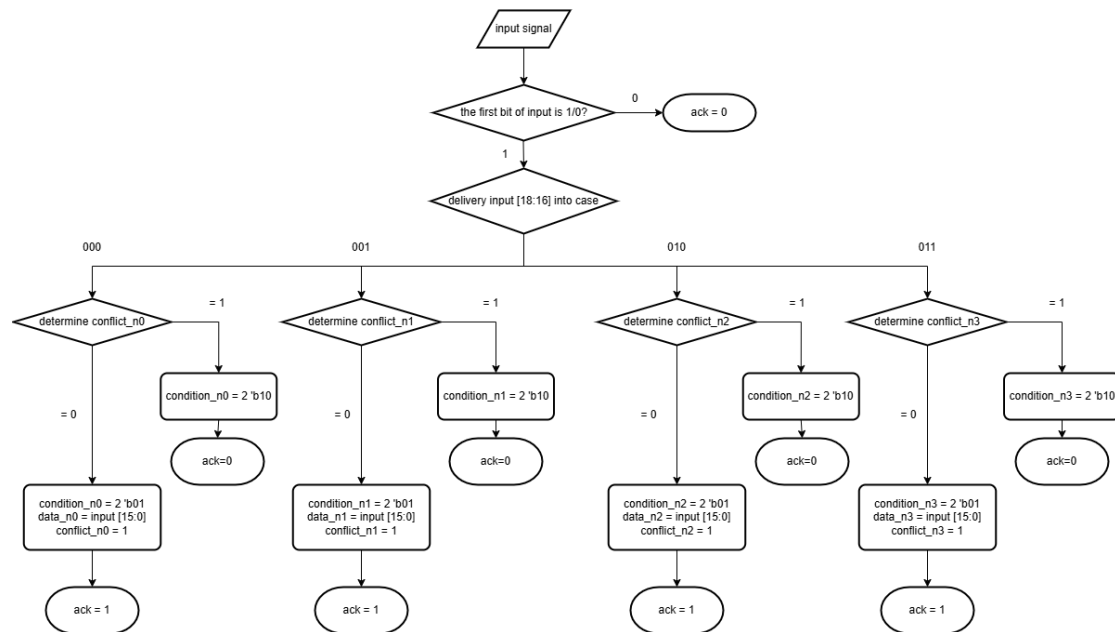
程式主要設計如下

```
if (in_n1[19] == 1) begin  
    case (in_n1[18:16])  
        3'b000: begin  
            if (conflict_n0 == 0) begin  
                condition_n0 = 2'b01;  
                data_n0 = in_n1[15:0];  
                conflict_n0 = 1;  
                ack_n1 = 1;  
            end else begin  
                condition_n0 = 2'b10;  
                ack_n1 = 0;  
            end  
        end  
    end
```

先判斷 in 最前的一個 bit 是否為 1，是才要操作，不是的話 ack 就直接為零。如果是 1 的話就開始進入 case 判斷 18~16 位置的編碼，然後用 2 進制解讀出傳輸 out 的代號。進入 case 當中後再分辨 conflict—確認是否為優先級，如果不是就 ack=0 和更新 condition=10(之前傳輸一次，這次再加一)，是的話就把 in 的資料

複製進去並更新 condition=01(out 第一次收到資料)、ack=1、conflict=1(表示這 out 已被佔住)。

2. 架構圖



3. 心得

在這次的作業中我確立的對 **verilog** 的興趣，沒想到除了常見的軟體語言之外硬體描述語言也那麼好玩，這次作業中的寫法讓我大開眼界，沒想到居然有點像 **python** 的儲存型態那樣，可以用範圍直接比較，說個題外話:與其說 **systemverilog** 像 **C++** 語法，我覺得更偏向 **python** 一些，不過感覺目前的手感還很陌生，預計之後會上網找資料自學，希望能在數位電路與系統這堂課中找到更多的樂趣。